Advanced Machine learning Mastering Course

Introduced by

George Samuel

Master in computer science
Cairo University

Innovisionray.com

2024

Agenda

1 Supervised Learning

2 Introduction to Regression

3 More Regression

4 Regression in Sklearn

5 Gradient decent-Normal equation

6 Regularization

7 Logistic Regression

8 Decision Tree

10 Neural Networks

11 Neural Nets Mini-Project

11 Math behind SVMs

11 SVMs in Practice

12 Instance Based Learning

13 Regression in Sklearn

14 Naive Bayes

15 Bayesian Inference

16 Ensemble B&B

17 Finding donors for CharityML

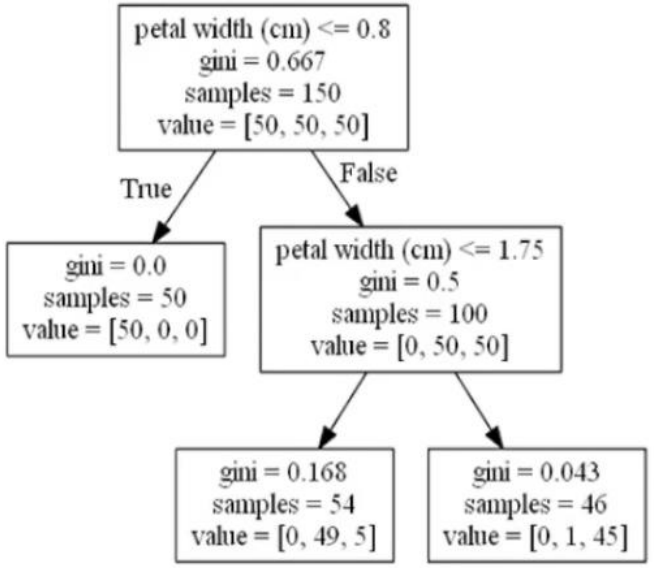
Agenda

- ➔ Classification Models - Decision Tree
- ➔ Ensemble Methods
- ➔ Classification Models - Random Forest
- ➔ Classification Models - Ensemble Methods (XGBoost)
- ➔ Implementation with Sklearn

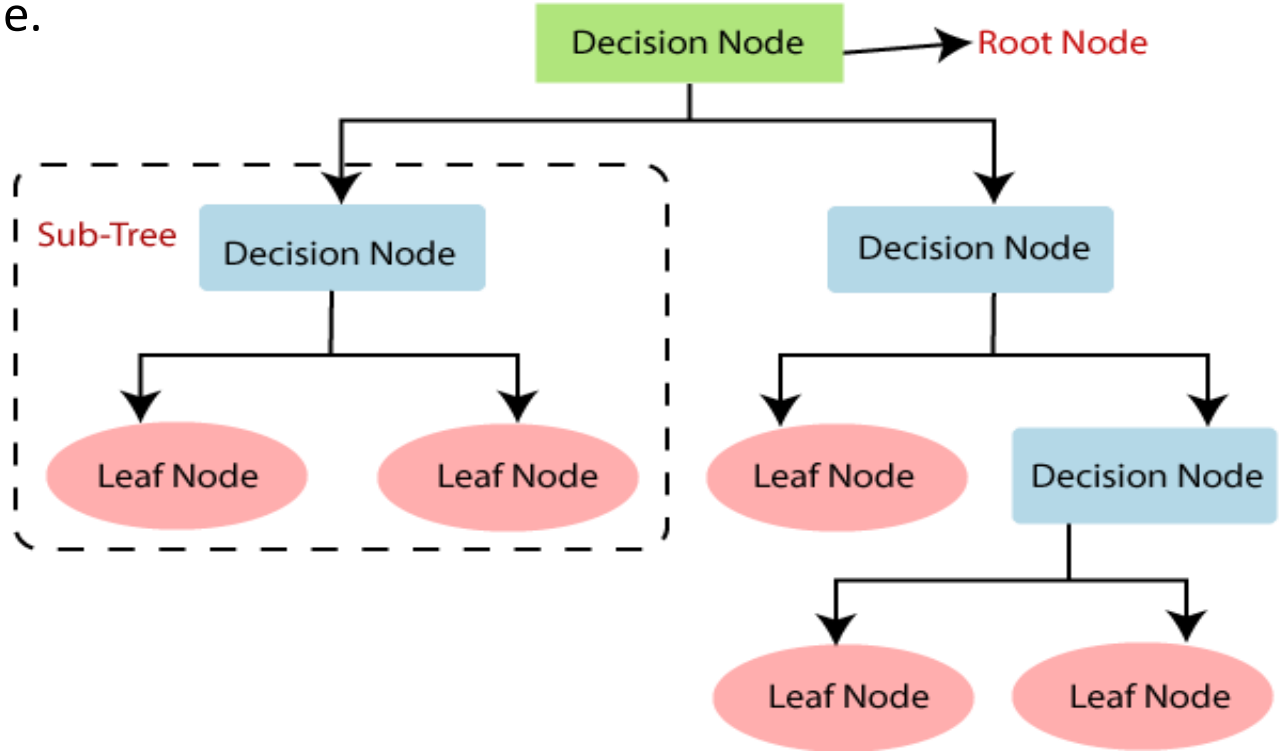
1. Classification Models - Decision Tree

Decision Tree Classification Algorithm:

➔ It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.



Decision tree built for the Iris Dataset



Decision Tree Classification Algorithm:

- In a **Decision Tree**, there are two nodes, which are the **Decision Node** and **Leaf Node**.
- Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
- It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.
- In order to build a tree, we use the **CART** algorithm, which stands for **Classification and Regression Tree** algorithm.

Decision Tree Terminologies:

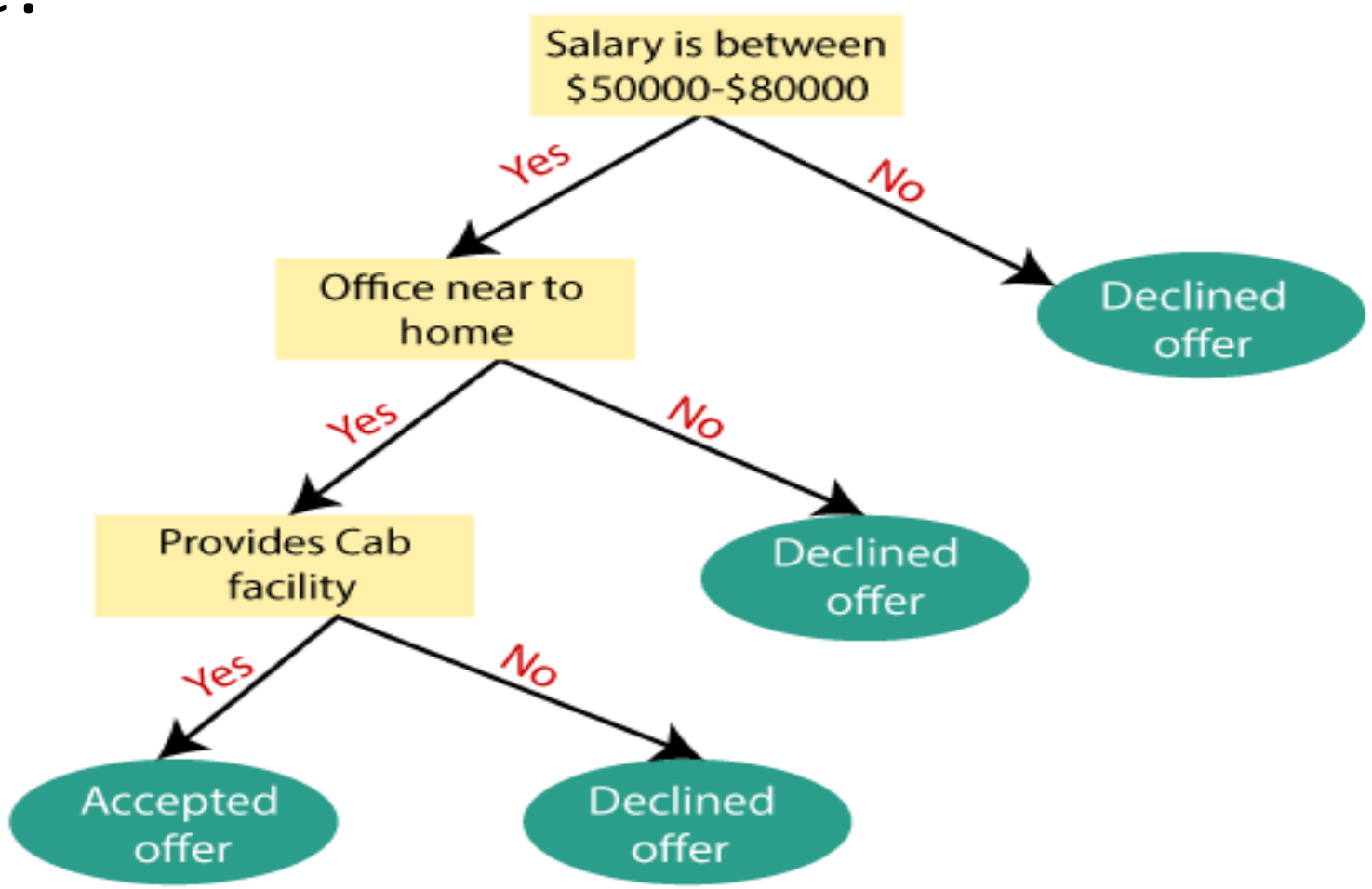
- **Root Node**: Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- **Leaf Node**: Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- **Splitting**: Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- **Branch/Sub Tree**: A tree formed by splitting the tree.
- **Parent/Child Node**: The root node of the tree is called the parent node, and other nodes are called the child nodes.

7

How Does the Decision Tree Algorithm Work?

- **Step-1**: Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2**: Find the best attribute in the dataset using **Attribute Selection Measure** (**ASM**).
- **Step-3**: Divide the S into subsets that contains possible values for the best attributes.
- **Step-4**: Generate the decision tree node, which contains the best attribute.
- **Step-5**: Recursively make new decision trees using the subsets of the dataset created in step-3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

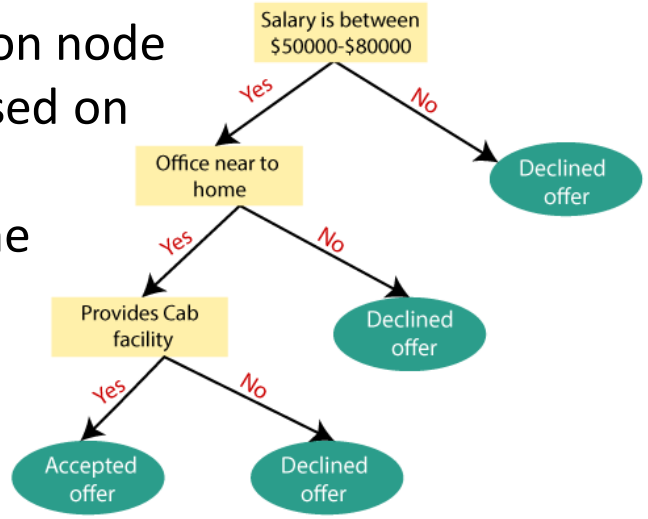
Example:


Salary is between $50000-$80000
Yes
No
Office near to home
Declined offer
Yes
No
Provides Cab facility
Declined offer
Yes
No
Accepted offer
Declined offer

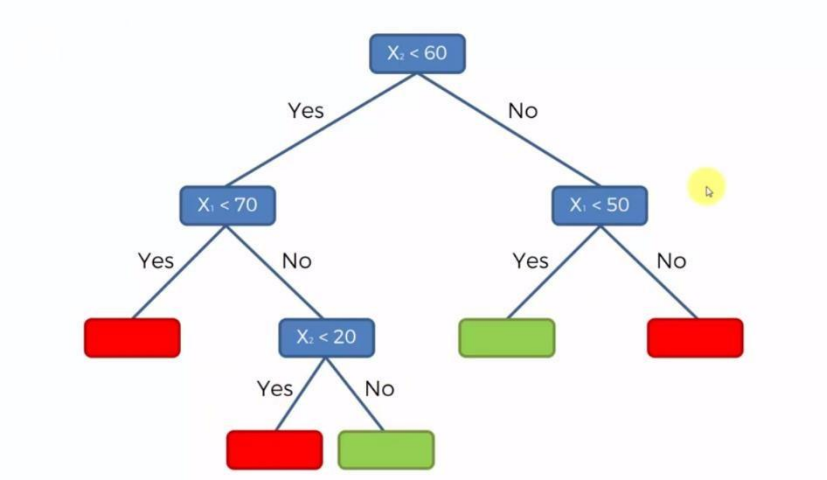
9

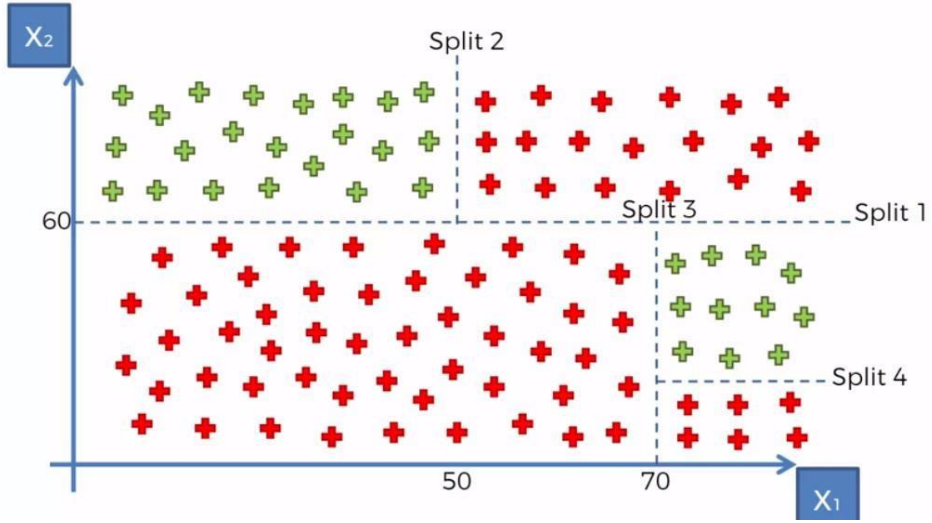
Example:

- ➔ Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not.
- ➔ So, to solve this problem, the decision tree starts with the root node (Salary attribute by **ASM**).
- ➔ The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels.
- ➔ The next decision node further gets split into one decision node (Cab facility) and one leaf node.
- ➔ Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer).



Example:


X₂ < 60
Yes
No
X₁ < 70
X₁ < 50
Yes
No
Yes
No
X₂ < 20
Yes
No


X2
Split 2
60
Split 3
Split 1
Split 4
50
70
X1

11

Attribute Selection Measures:

- While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes.
- So, to solve such problems there is a technique which is called as Attribute selection measure **(ASM)**.
- By this measurement, we can easily select the best attribute for the nodes of the tree.
- There are two popular techniques for ASM, which are:
  - **Information Gain**
  - **Gini Index**

Information Gain:

- **Information Gain**: It is a metric used to train Decision Trees. Specifically, this metric measure the quality of a split.
- **Entropy**: It is a metric to measure the impurity in a given attribute. It specifies randomness in data.

$$E = -\sum_{i}^{C} p_i \log_2 p_i$$

Entropy = - [P(yes) log2 P(yes) + P(no) log2 P(no)]

Where ”P” is the probability of randomly picking an element of class i (i.e. the proportion of the dataset made up of class i)

Information Gain:

- **Information Gain** is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first.

Information Gain = Entropy(data) - [(Weighted Avg) * Entropy(each feature)]

Information Gain - Entropy Example:

The easiest way to understand this is with an example. Consider a dataset with 1 blue, 2 greens, and 3 reds: ●●●●●●. Then

$$E = -(p_b \log_2 p_b + p_g \log_2 p_g + p_r \log_2 p_r)$$

We know $p_b = \frac{1}{6}$ because $\frac{1}{6}$ of the dataset is blue. Similarly, $p_g = \frac{2}{6}$ (greens) and $p_r = \frac{3}{6}$ (reds). Thus,

$$\begin{aligned} E &= -(\frac{1}{6}\log_2(\frac{1}{6}) + \frac{2}{6}\log_2(\frac{2}{6}) + \frac{3}{6}\log_2(\frac{3}{6})) \\ &= \boxed{1.46} \end{aligned}$$

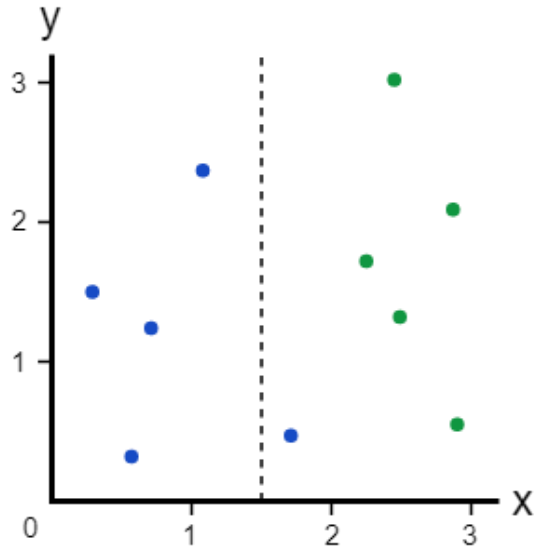
What about a dataset of all one color? Consider 3 blues as an example: ●●●. The entropy would be

$$E = -(1\log_2 1) = \boxed{0}$$

Information Gain - Complete Example:

Before the split, we had 5 blues and 5 greens, so the entropy was

$$E_{before} = -(0.5\log_2 0.5 + 0.5\log_2 0.5)$$
$$= \boxed{1}$$



After the split, we have two branches.

Left Branch has 4 blues, so $E_{left} = \boxed{0}$ because it's a dataset of all one color.

Right Branch has 1 blue and 5 greens, so

$$E_{right} = -(\frac{1}{6}\log_2(\frac{1}{6}) + \frac{5}{6}\log_2(\frac{5}{6}))$$
$$= \boxed{0.65}$$

Information Gain - Complete Example:

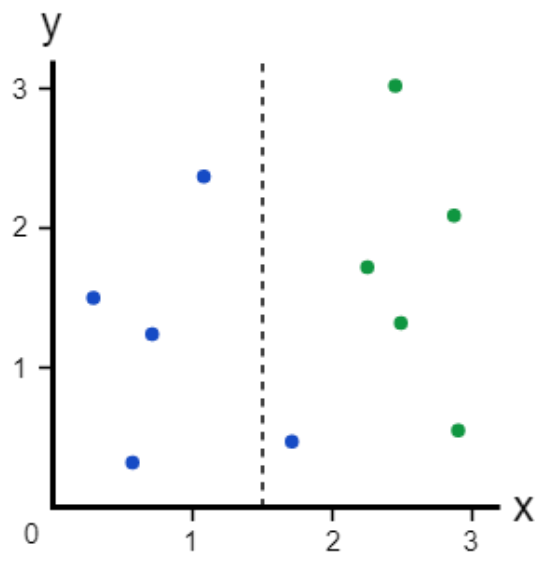
Now that we have the entropies for both branches, we can determine the quality of the split by **weighting the entropy of each branch by how many elements it has**. Since Left Branch has 4 elements and Right Branch has 6, we weight them by 0.4 and 0.6, respectively:

$$\begin{aligned} E_{split} &= 0.4 * 0 + 0.6 * 0.65 \\ &= \boxed{0.39} \end{aligned}$$

We started with $E_{before} = 1$ entropy before the split and now are down to 0.39! **Information Gain = how much Entropy we removed**, so

$$\text{Gain} = 1 - 0.39 = \boxed{0.61}$$

This makes sense: **higher Information Gain = more Entropy removed**, which is what we want. In the perfect case, each branch would contain only one color after the split, which would be zero entropy!



Gini Index/Impurity:

➔ **Gini Impurity:** It is the probability of incorrectly classifying a randomly chosen element in the dataset if it was randomly labelled according to the class distribution in the dataset.

$$G = \sum_{i=1}^{C} p(i) * (1 - p(i))$$

where C is the number of classes and $p(i)$ is the probability of randomly picking an element of class i.

When training a decision tree, the best split is chosen by **maximizing the Gini Gain**, which is calculated by subtracting the weighted impurities of the branches from the original impurity.

Gini Index/Impurity:

- **Gini Index** is a measure of impurity used while creating a decision tree in the CART algorithm.
- An attribute with the low Gini Index should be preferred as compared to the high Gini Index.
- It only creates **binary splits**, and the CART algorithm uses the Gini Index to create binary splits.

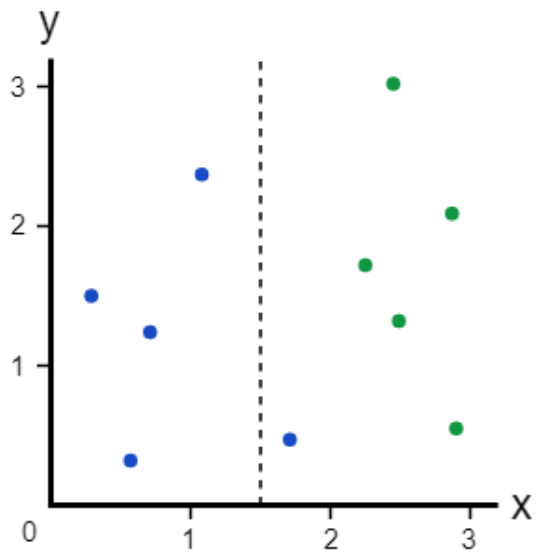
Gini Index - Complete Example:

If we have C total classes and $p(i)$ is the probability of picking a datapoint with class i, then the Gini Impurity is calculated as

$$G = \sum_{i=1}^{C} p(i) * (1 - p(i))$$

For the example above, we have $C = 2$ and $p(1) = p(2) = 0.5$, so

$$\begin{aligned} G &= p(1) * (1 - p(1)) + p(2) * (1 - p(2)) \\ &= 0.5 * (1 - 0.5) + 0.5 * (1 - 0.5) \\ &= \boxed{0.5} \end{aligned}$$

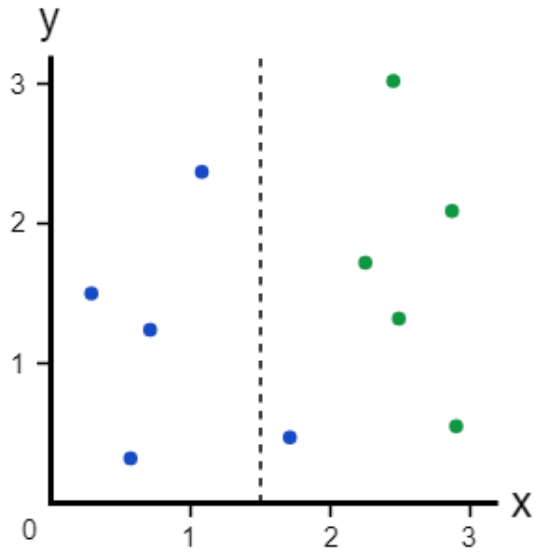


Gini Index - Complete Example:

Left Branch has only blues, so we know that $G_{left} = \boxed{0}$.

Right Branch has 1 blue and 5 greens, so

$$
\begin{aligned}
G_{right} &= \frac{1}{6} * (1 - \frac{1}{6}) + \frac{5}{6} * (1 - \frac{5}{6}) \\
&= \frac{5}{18} \\
&= \boxed{0.278}
\end{aligned}
$$



Gini Index - Complete Example:

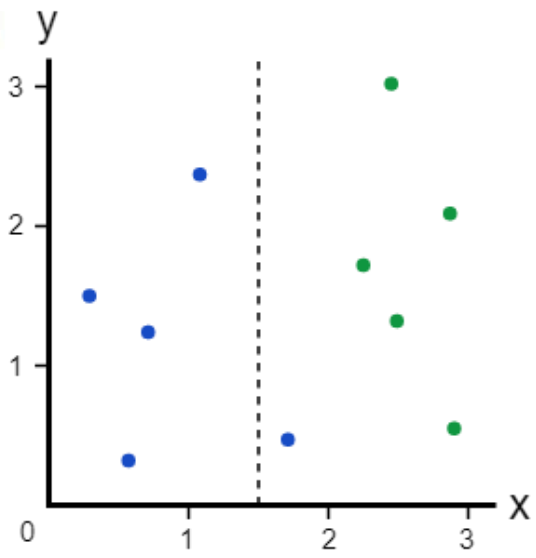
We'll determine the quality of the split by **weighting the impurity of each branch by how many elements it has**. Since Left Branch has 4 elements and Right Branch has 6, we get:

$$(0.4 * 0) + (0.6 * 0.278) = 0.167$$

Thus, the amount of impurity we've "removed" with this split is

$$0.5 - 0.167 = \boxed{0.333}$$

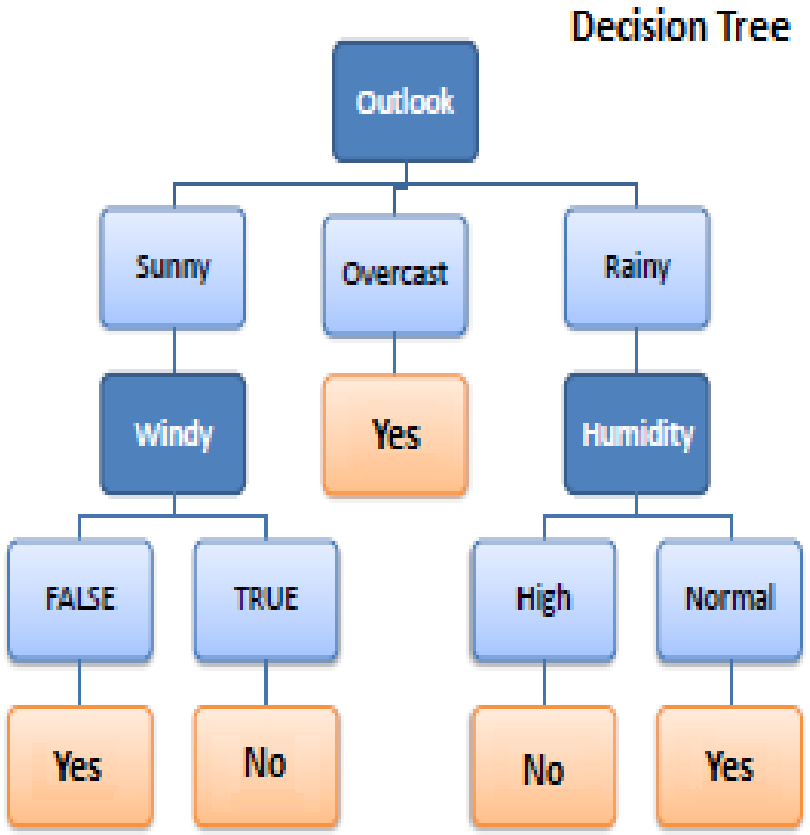
I'll call this value the Gini Gain. This is what's used to pick the best split in a decision tree! **Higher Gini Gain = Better Split**.



Decision Tree ID3 Algorithm

Predictors | Target

Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



Decision Tree ID3 Algorithm

Build Decision Tree Steps USING ID3 Algorithm :

Step 1: Calculate entropy of the target Label .

Step 2: The dataset is then split on the different attributes. Calculate entropy and information gain for each attributes (Column)

Step 3: Choose attribute with the largest information gain as the decision node, divide the dataset by its branches and repeat the same process on every branch

Step 4: A branch with entropy of 0 is a leaf node. And A branch with entropy more than 0 needs further splitting

Step 5: The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified

Decision Tree ID3 Algorithm

Step 1: Calculate entropy of the target Label .

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5

Entropy(PlayGolf) = Entropy (5,9)

= Entropy (0.36, 0.64)

= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)

= 0.94

Decision Tree ID3 Algorithm

Step 2: The dataset is then split on the different attributes. Calculate entropy and information gain for each attributes (Column)

$$E(T,X) = \sum_{c \in X} P(c)E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

E(PlayGolf, Outlook) = **P**(Sunny)***E**(3,2) + **P**(Overcast)***E**(4,0) + **P**(Rainy)***E**(2,3)

= (5/14)*0.971 + (4/14)*0.0 + (5/14)*0.971

= 0.693

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
Gain = 0.029			

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
Gain = 0.152			

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
Gain = 0.048			

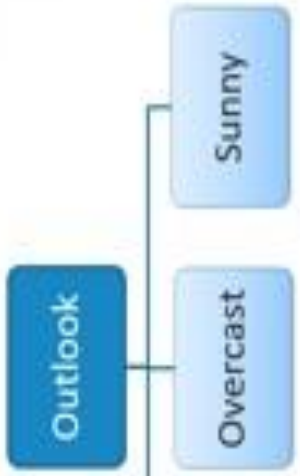
$$Gain(T,X) = Entropy(T) - Entropy(T,X)$$

G(PlayGolf, Outlook) = **E**(PlayGolf) – **E**(PlayGolf, Outlook)

= 0.940 – 0.693 = 0.247

Decision Tree ID3 Algorithm

Step 3: Choose attribute with the largest information gain as the decision node, divide the dataset by its branches and repeat the same process on every branch



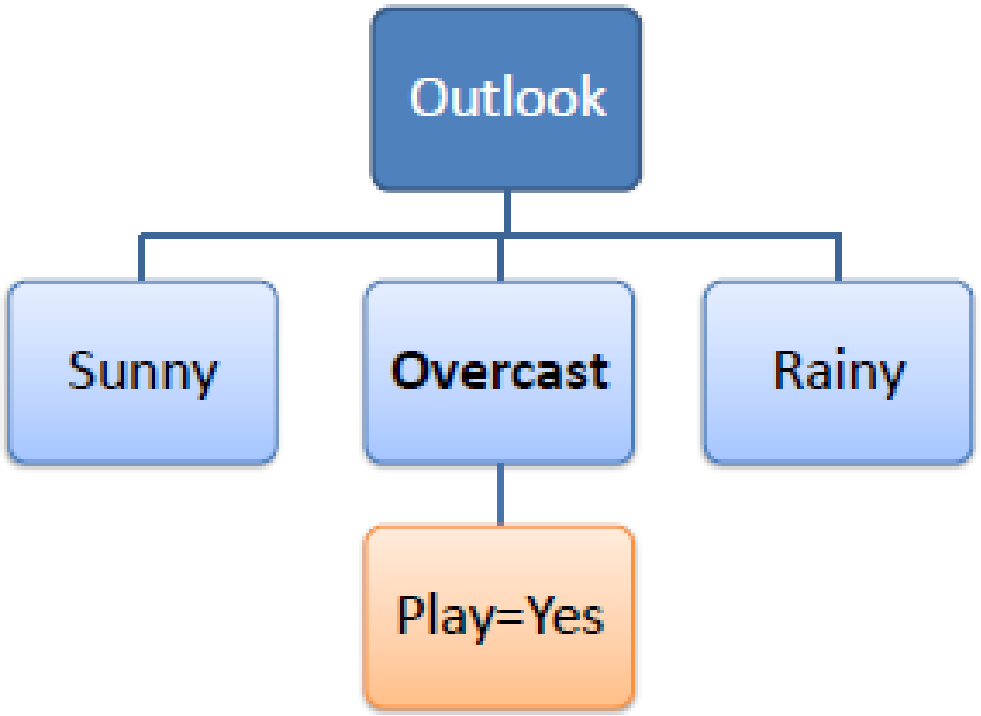
Outlook	Temp	Humidity	Windy	Play Golf
Sunny	Mild	High	FALSE	Yes
Sunny	Cool	Normal	FALSE	Yes
Sunny	Cool	Normal	TRUE	No
Sunny	Mild	Normal	FALSE	Yes
Sunny	Mild	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
Overcast	Cool	Normal	TRUE	Yes
Overcast	Mild	High	TRUE	Yes
Overcast	Hot	Normal	FALSE	Yes
Rainy	Hot	High	FALSE	No
Rainy	Hot	High	TRUE	No
Rainy	Mild	High	FALSE	No
Rainy	Cool	Normal	FALSE	Yes
Rainy	Mild	Normal	TRUE	Yes

★		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

Decision Tree ID3 Algorithm

Step 4a: A branch with entropy of 0 is a leaf node.

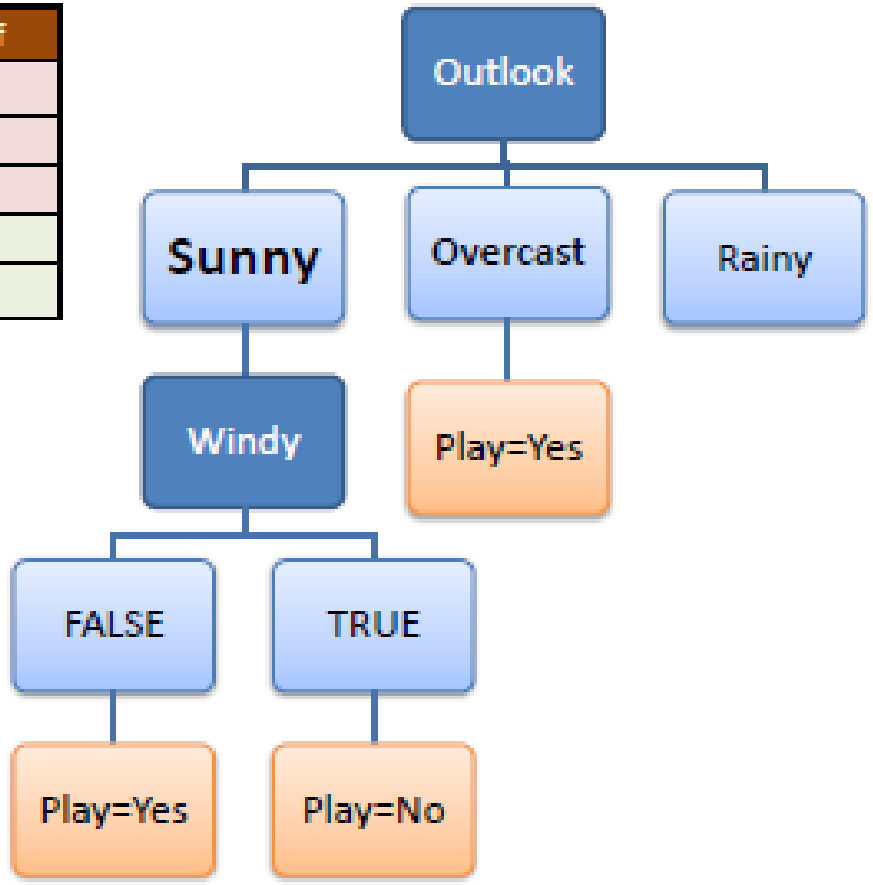
Temp	Humidity	Windy	Play Golf
Hot	High	FALSE	Yes
Cool	Normal	TRUE	Yes
Mild	High	TRUE	Yes
Hot	Normal	FALSE	Yes



Decision Tree ID3 Algorithm

Step 4b: A branch with entropy more than 0 needs further splitting.

Temp.	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



Decision Tree ID3 Algorithm

Step 5: The ID3 algorithm is run recursively on the non-leaf branches, until all data is classified

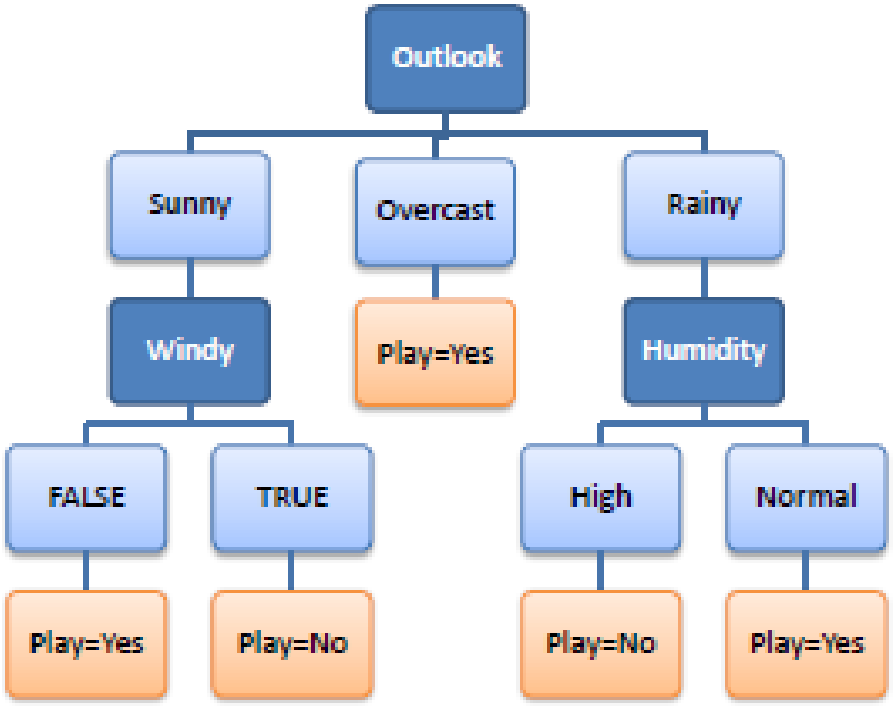
R_1: IF (Outlook=Sunny) AND (Windy=FALSE) THEN Play=Yes

R_2: IF (Outlook=Sunny) AND (Windy=TRUE) THEN Play=No

R_3: IF (Outlook=Overcast) THEN Play=Yes

R_4: IF (Outlook=Rainy) AND (Humidity=High) THEN Play=No

R_5: IF (Outlook=Rain) AND (Humidity=Normal) THEN Play=Yes



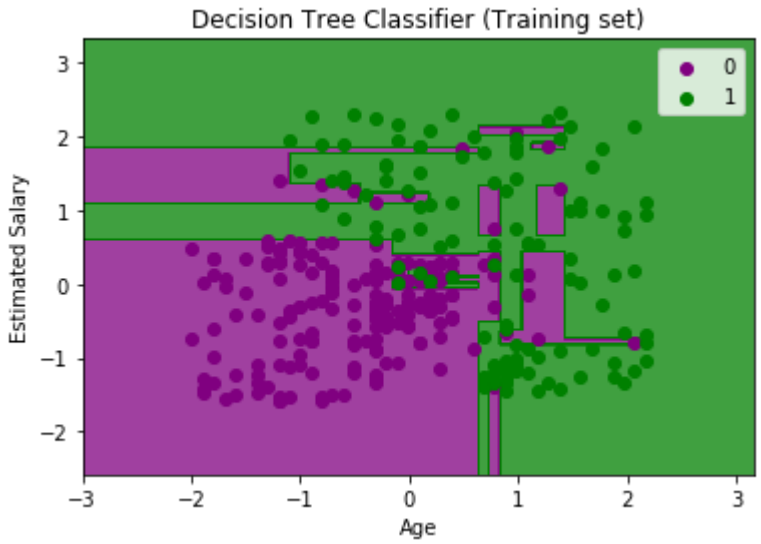
Advantages of the Decision Tree:

- ➔ It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- ➔ It can be very useful for solving decision-related problems.
- ➔ It helps to think about all the possible outcomes for a problem.
- ➔ There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree:

- ➔ The decision tree contains lots of layers, which makes it complex.
- ➔ It may have an overfitting issue, which can be resolved using the Random Forest algorithm.
- ➔ For more class labels, the computational complexity of the decision tree may increase.

Sklearn Decision Tree:



➔ The output is completely different from the rest classification models. It has both vertical and horizontal lines that are splitting the dataset according to the age and estimated salary variables.

2. Ensemble Methods

Ensemble learning

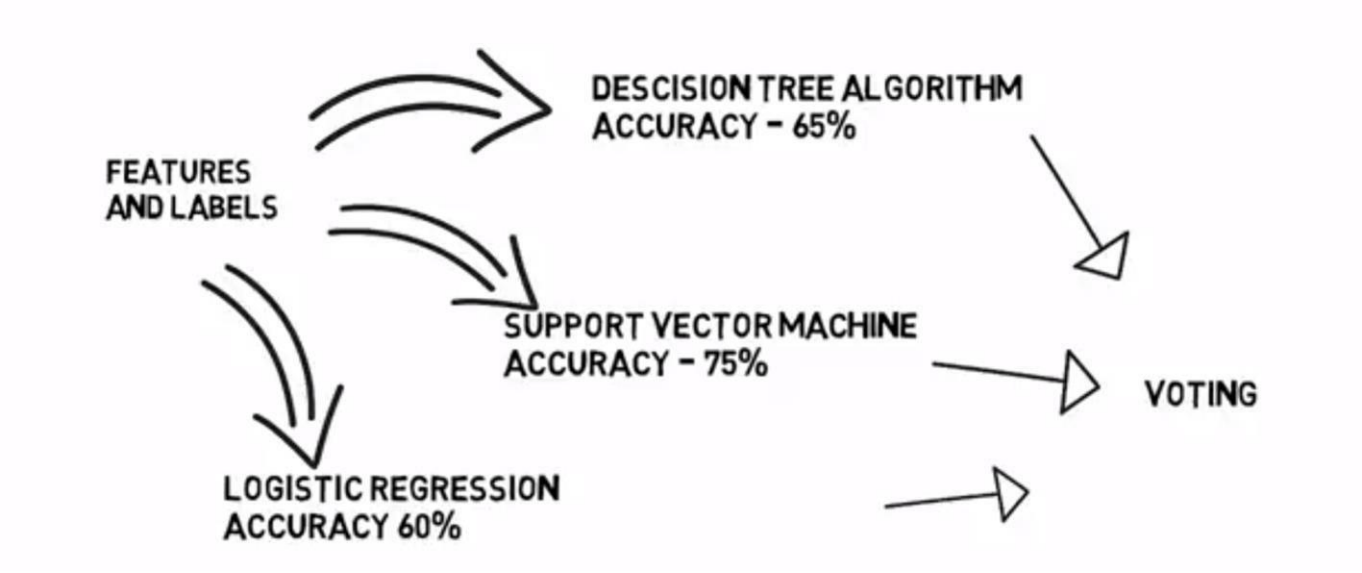
Ensemble techniques

1- Bagging (bootstrap aggregation)
One type :- Random forest

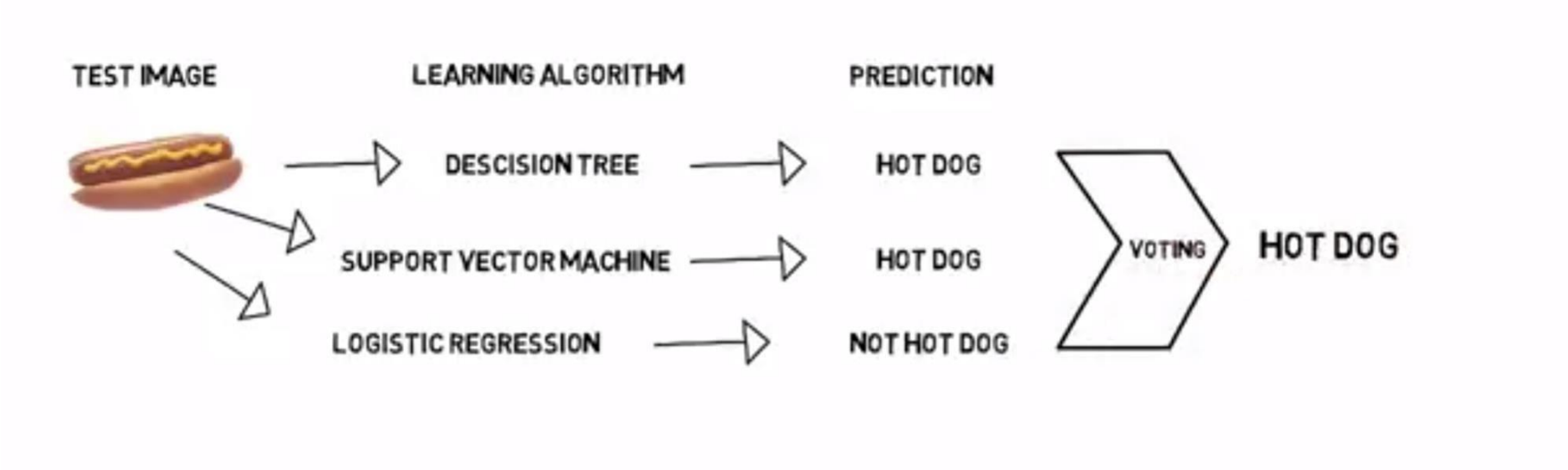
2- Boosting
.Ada boost
.Gradient boosting
.XGboost

Ensemble Learning:

- ➔ Using multiple learning algorithms together for the same task.
- ➔ Better predictions than individual learning algorithms.



Ensemble Learning:



Ensemble Learning:

➔ Why use Ensemble Methods?

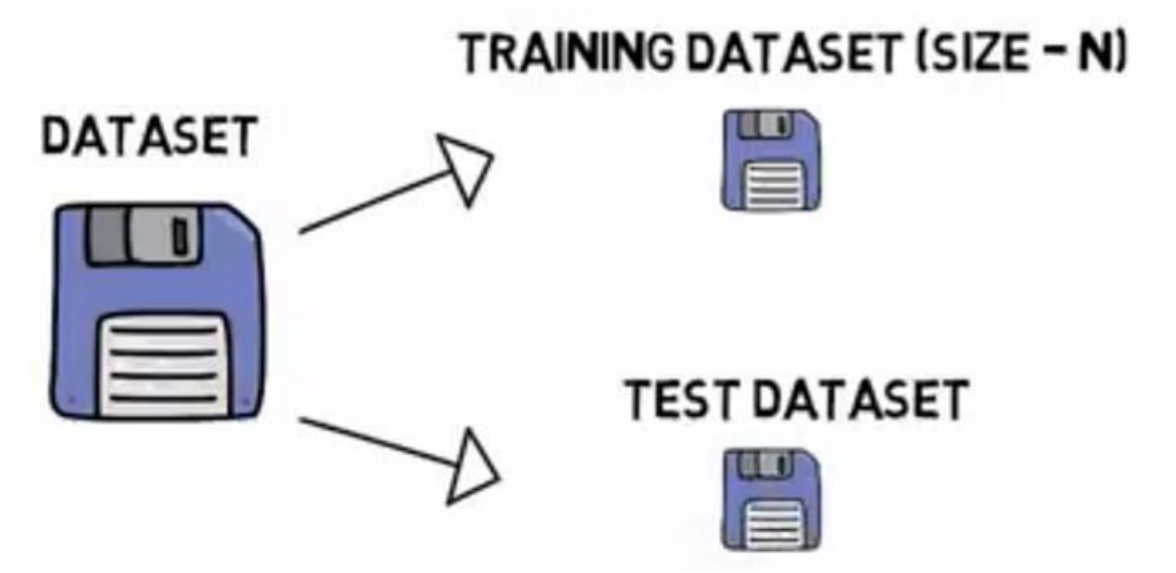
- Better accuracy (lower error).
- Higher consistency (avoid over fitting).
- Reduce bias and variance errors.

➔ When and Where to use Ensemble Methods?

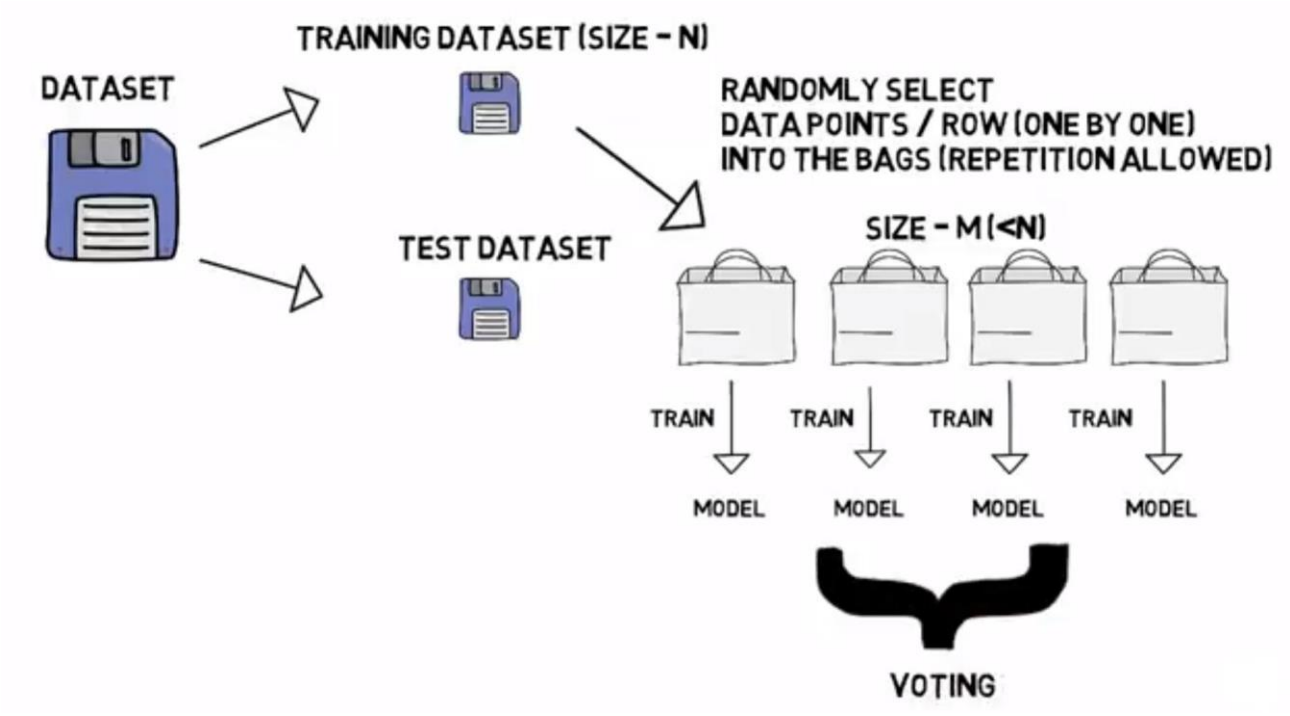
- Single model over fits.
- Results worth the extra training.
- Can be used for classification and regression.

Bootstrap Aggregating (Bagging):

➔ Multiple models of same learning algorithm trained with subsets of dataset randomly picked from the training dataset.

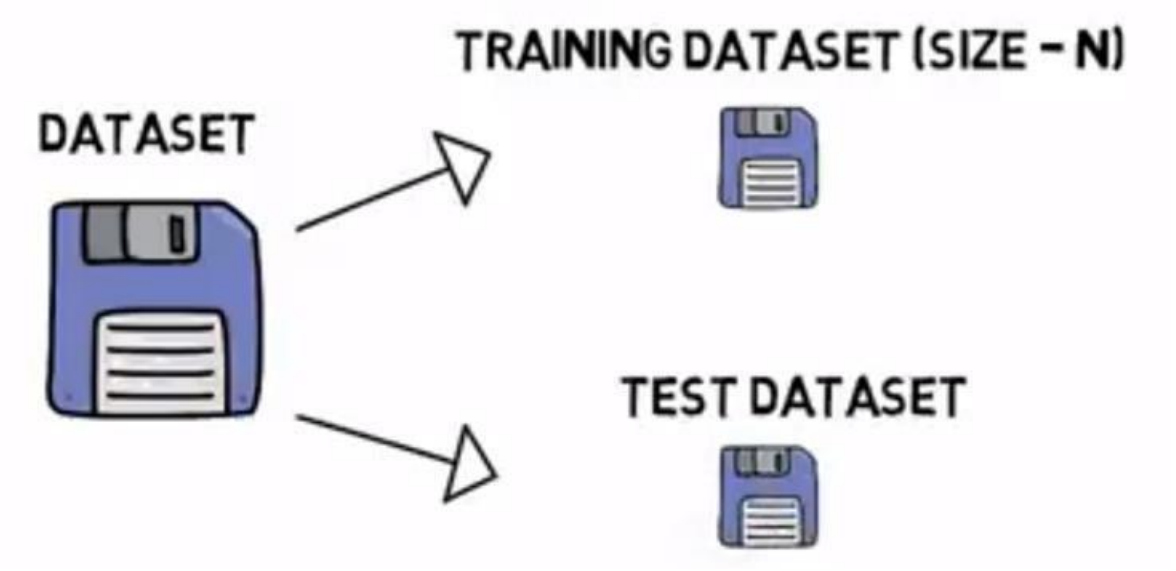


Bootstrap Aggregating (Bagging):

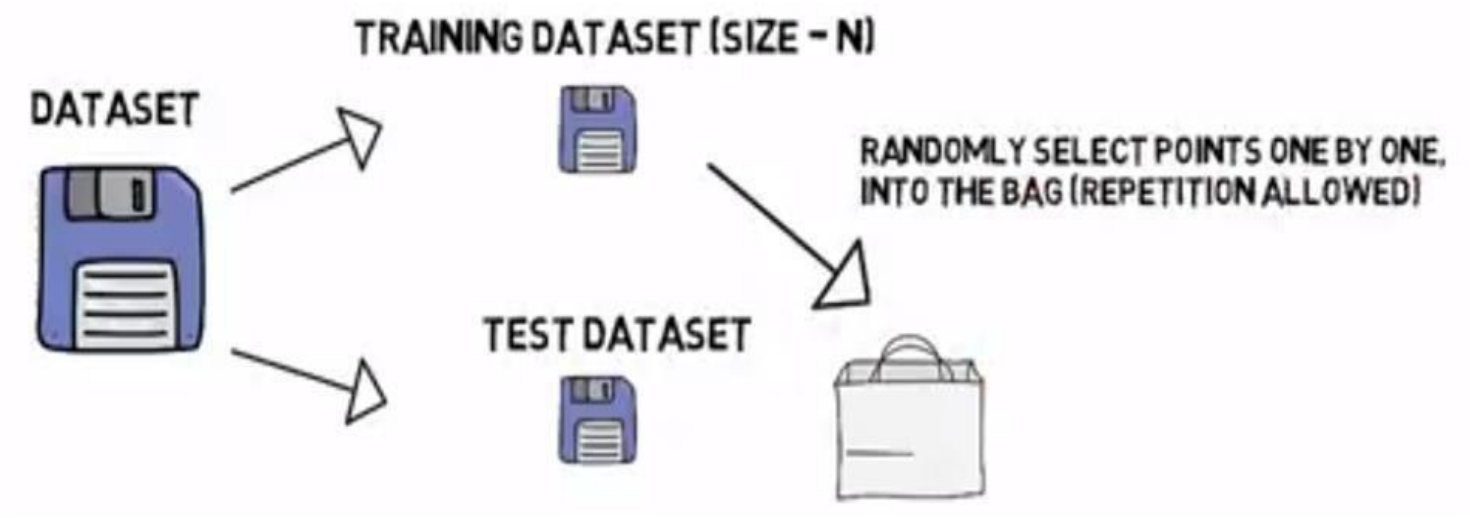


Boosting:

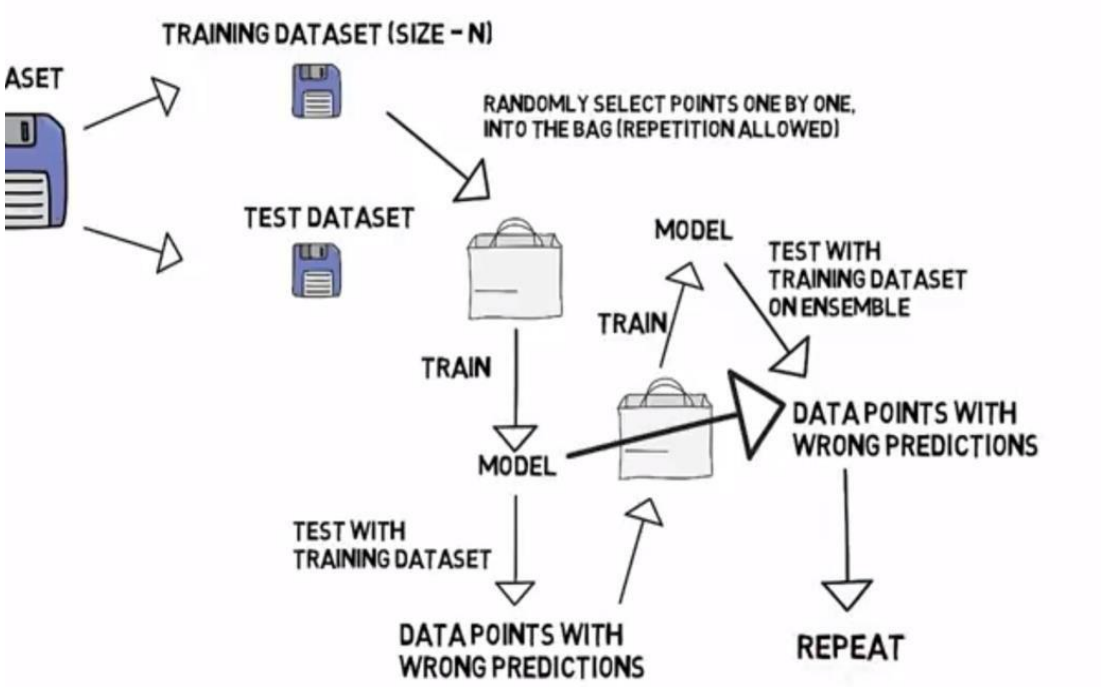
- ➔ Little variation on Bagging.
- ➔ Selecting points which give wrong predictions.

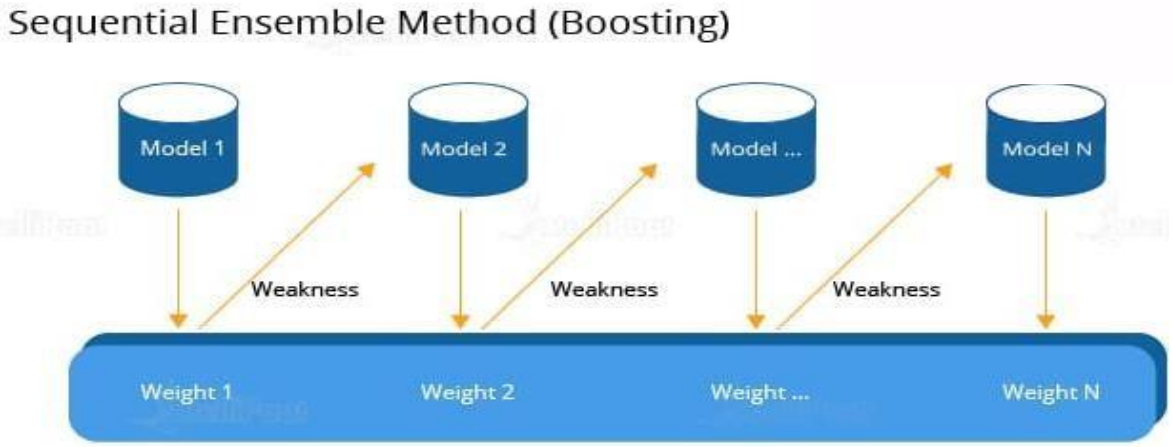


Boosting:

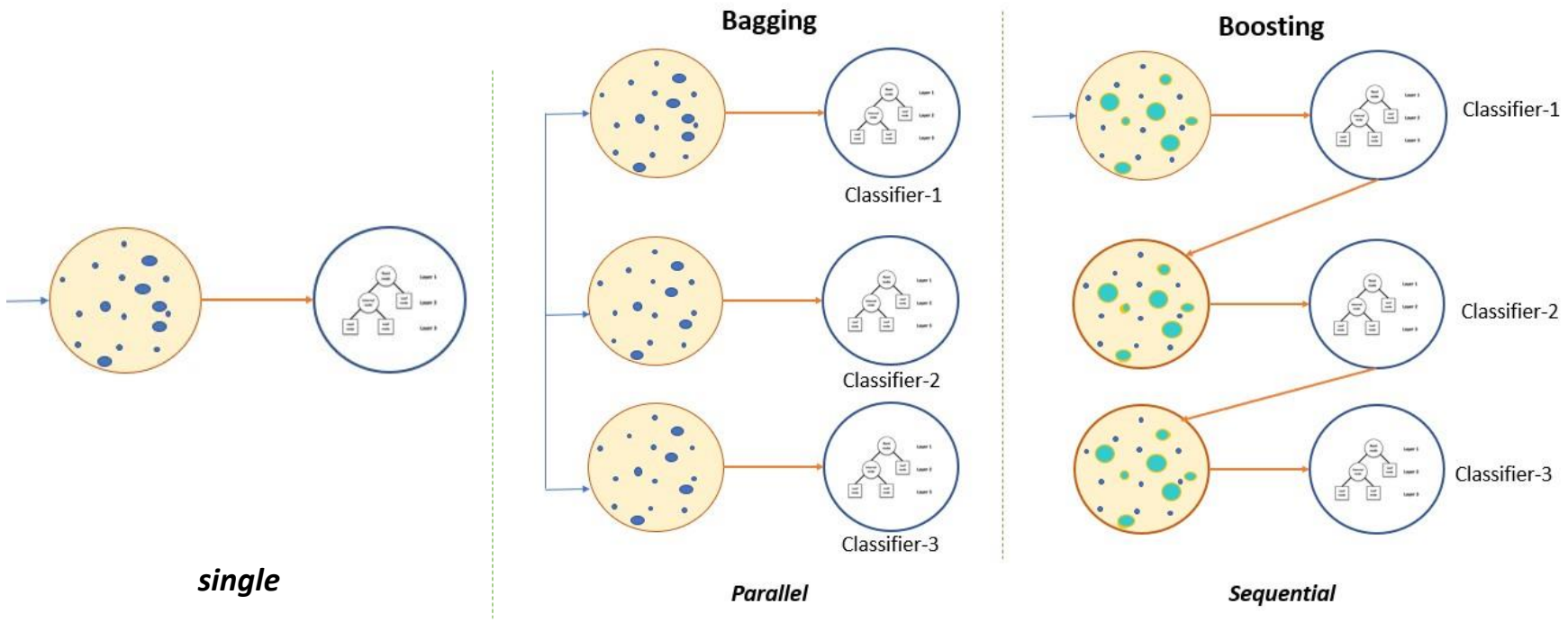

TRAINING DATASET (SIZE - N)
DATASET
RANDOMLY SELECT POINTS ONE BY ONE,
INTO THE BAG (REPETITION ALLOWED)
TEST DATASET

Boosting:


TRAINING DATASET (SIZE – N)
ASET
RANDOMLY SELECT POINTS ONE BY ONE, INTO THE BAG (REPETITION ALLOWED)
TEST DATASET
MODEL
TEST WITH TRAINING DATASET ON ENSEMBLE
TRAIN
TRAIN
DATA POINTS WITH WRONG PREDICTIONS
MODEL
TEST WITH TRAINING DATASET
DATA POINTS WITH WRONG PREDICTIONS
REPEAT


Sequential Ensemble Method (Boosting)
Model 1
Model 2
Model ...
Model N
Weakness
Weakness
Weakness
Weight 1
Weight 2
Weight ...
Weight N

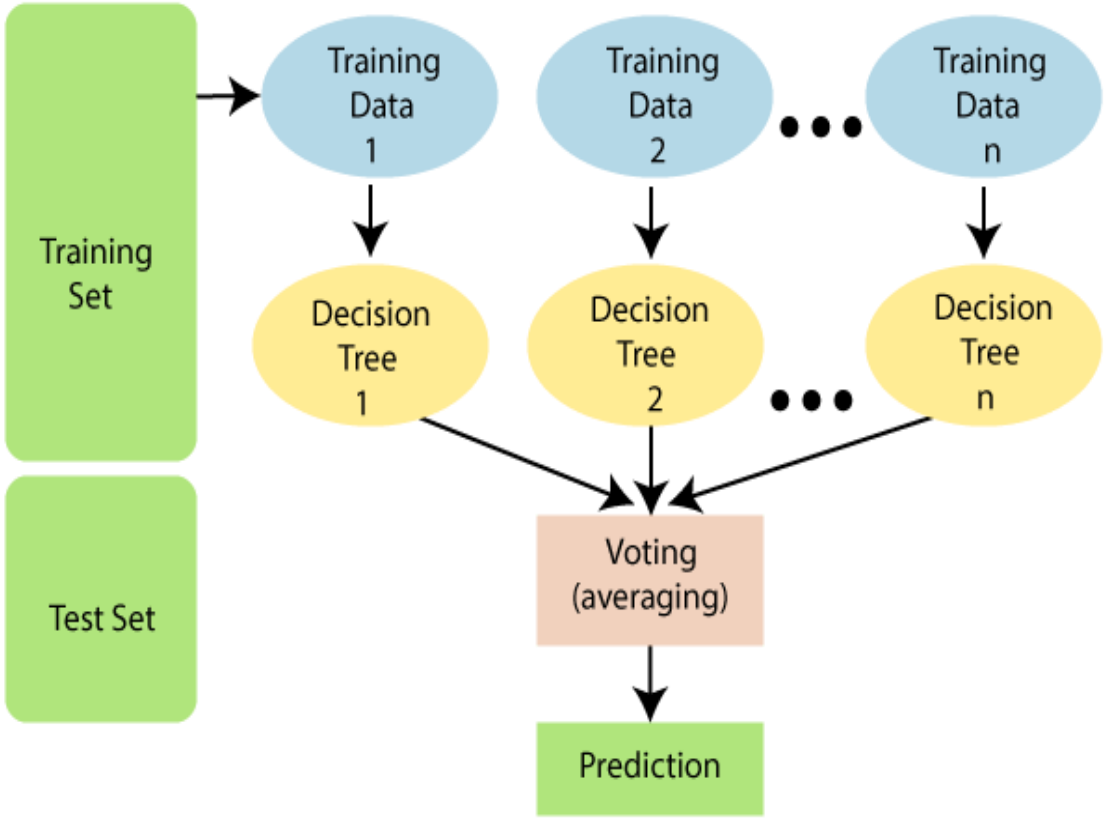
Ensemble:


Bagging
Boosting
Classifier-1
Classifier-2
Classifier-3
Classifier-1
Classifier-2
Classifier-3
single
Parallel
Sequential

3. Classification Models - Random Forest

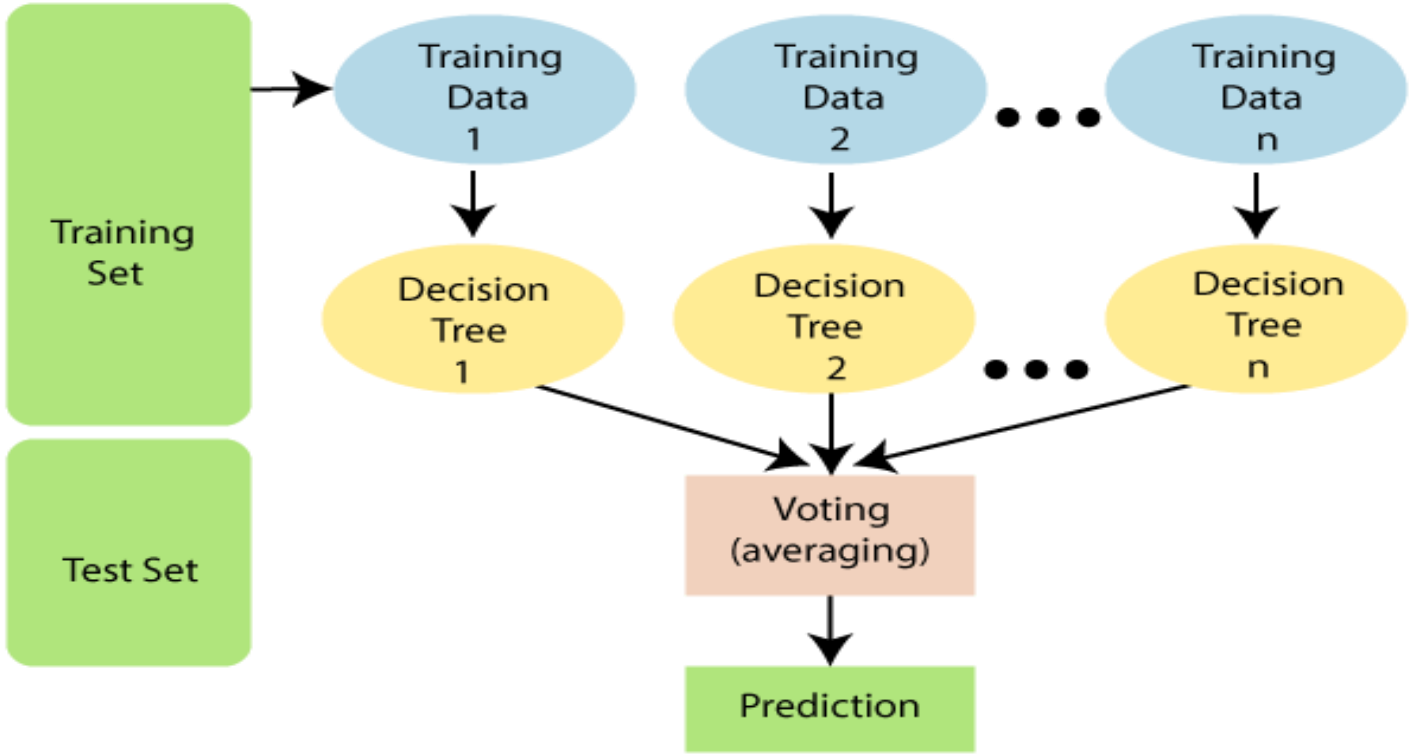
Random Forest Classification Algorithm:

➔ Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.



Random Forest Classification Algorithm:

- ➔ Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, it predicts the final output.
- ➔ The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.



How Does the Random Forest Algorithm Work?

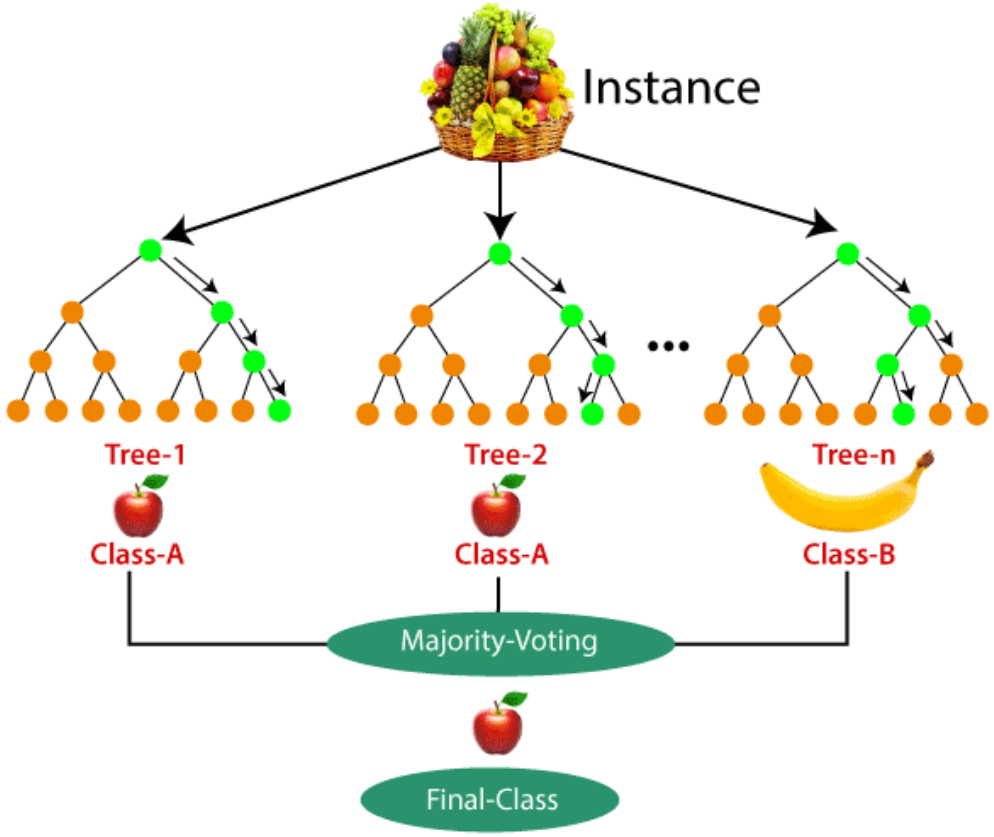
Random Forest works in two-phase:

- ➔ First is to create the random forest by combining N decision tree.
- ➔ And second is to make predictions for each tree created in the first phase.

The Working process can be explained in the below steps:

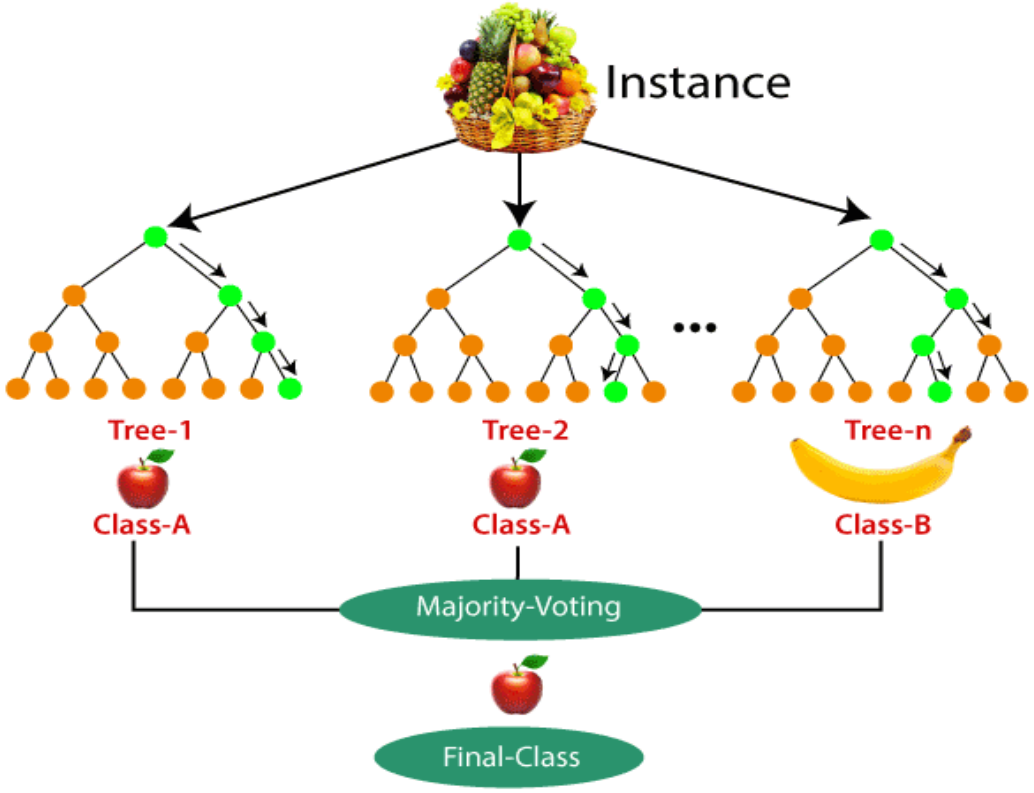
- ➔ **Step-1**: Select random K data points from the training set.
- ➔ **Step-2**: Build the decision trees associated with the selected data points.
- ➔ **Step-3**: Choose the number N for decision trees that you want to build.
- ➔ **Step-4**: Repeat Step 1 & 2.
- ➔ **Step-5**: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

Example:


Instance
Tree-1
Tree-2
Tree-n
Class-A
Class-A
Class-B
Majority-Voting
Final-Class

Example:

- ➔ Suppose there is a dataset that contains multiple fruit images.
- ➔ So, this dataset is given to the Random forest classifier.
- ➔ The dataset is divided into subsets and given to each decision tree.
- ➔ During the training phase, each decision tree produces a prediction result.
- ➔ And when a new data point occurs, then based on the majority of results, the Random Forest classifier predicts the final decision.



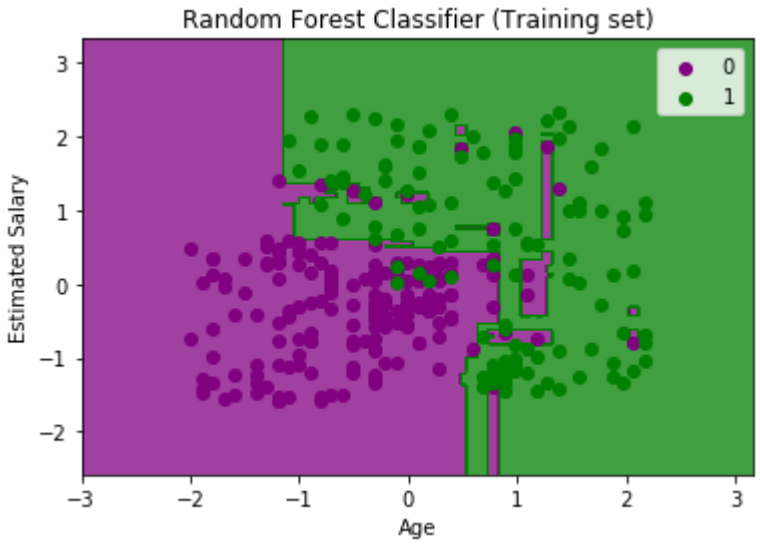
Advantages of the Random Forest:

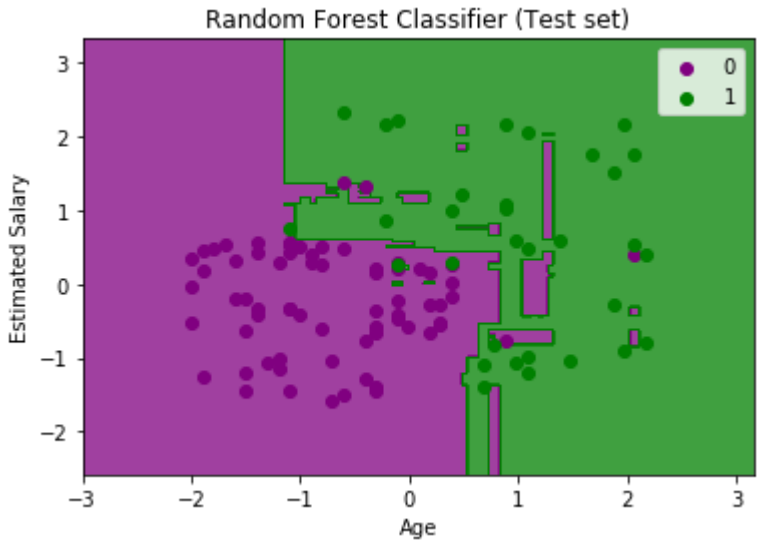
- ➔ Random Forest is capable of performing both Classification and Regression tasks.
- ➔ It is capable of handling large datasets with high dimensionality.
- ➔ It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages of the Random Forest:

➔ Although Random Forest can be used for both classification and regression tasks, it is not more suitable for Regression tasks.

Sklearn Random Forest:





Boost

THANK YOU!